FilmNote (FD&Ceci) 技术总结

2026-05-04 | 单体 SPA | 3300 行 | GitHub Pages + Supabase + Cloudflare Workers

一、项目概述

FilmNote 是 Fank1ng 与 ceci 二人共用的电影/剧集评分系统。支持多用户、实时同步、个性化推荐、六维评分和统计分析。

技术栈：原生 HTML/CSS/JS（无框架） + Supabase BaaS + Cloudflare Workers

运行成本：$0/月（全部在免费额度内）

二、架构图

```
┌──────────────────────────────────────────────────────────┐
│                        GitHub Pages                      │
│  ┌──────────────────────────────────────────────────┐    │
│  │              index.html (3300 行 SPA)            │    │
│  │  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────────┐       │    │
│  │  │ 登录 │ │ 影单 │ │ 发现 │ │ 统计分析 │       │    │
│  │  │ 注册 │ │ 搜索 │ │ 推荐 │ │ 柱状图   │       │    │
│  │  │ 认证 │ │ 筛选 │ │ 热门 │ │ 维度对比 │       │    │
│  │  └──────┘ └──────┘ └──────┘ └──────────┘       │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
            │                          │
            ▼                          ▼
   ┌──────────────┐          ┌──────────────────┐
   │ Supabase     │          │ Cloudflare       │
   │  (BaaS)      │          │  Worker          │
   │              │          │                  │
   │ · Auth       │          │ /tmdb/*   代理   │
   │ · DB+RLS     │          │ /recommend 推荐  │
   │ · Realtime   │          │ /health   健康   │
   └──────────────┘          └──────────────────┘
                                     │
                                     ▼
                              ┌──────────────┐
                              │  TMDB API v3 │
                              └──────────────┘
```

```
│   （电影数据源）        │
└──────────────────────┘
```

三、核心功能清单

3.1 用户系统

功能	实现
邮箱注册/登录	Supabase Auth
邀请码控制	3 小时有效期，fank1ng/ceci 可见管理按钮
忘记/重置/修改密码	Supabase Auth 邮件流
用户颜色系统	fank1ng 金(#d4a853) / ceci 粉(#FF69B4) / 其他 蓝(#5b9db0)
显示名设置	Supabase profiles 表，Realtime 同步

3.2 影单（核心模块）

功能	说明
电影/剧集切换	两个子标签
所有者筛选	全部 / fank1ng / ceci
搜索	中英文标题搜索，350ms 防抖
排序	评分/年份/最近添加
分页	20 条/页，最多显示 5 个页码按钮
合并卡片	特权用户显示自身+伴侣+平均；普通用户显示自身+fank1ng+ceci
快捷评分	弹窗式六维评分，无需跳转
详情弹窗	TMDB 简介/导演/演员/评分/类型
编辑/删除	自己的条目可编辑（弹窗快捷编辑）或删除

防重复评价	同一用户同一电影/剧集不可重复添加

3.3 添加评价

功能	说明
TMDB 搜索	输入关键词自动搜索，选择后自动填入标题/年份/海报/导演
类型切换	电影/剧集
六维滑块评分	story(25%) / character(20%) / visual(15%) / editing(10%) / sound(10%) / emotion(20%)
总评分自动计算	加权求和
评语	可选
手动输入	支持无 TMDB 条目的手动录入

3.4 发现页（推荐引擎）

功能	说明
Ceci/FD 为你推荐	个性化推荐算法（见第四章）
本周热门	TMDB trending API 热度 Top12
Top100	TMDB top_rated，vote_count≥500 筛选，"只看未看"按用户隔离
快捷评分	推荐卡片一键打分
详情弹窗	点击卡片查看完整 TMDB 信息
刷新冷却	5 秒

3.5 统计分析

功能	说明

三视图切换	个人 / 他人 / 对比
分数分布柱状图	flexbox 实现，百分比高度
六维平均对比	维度标签带用户色
统计数值	颜色跟随用户色

3.6 实时同步

- Supabase Realtime 监听 entries 表变更
- 其他人添加/修改评分时自动更新当前页面
- CLOSED 状态 3 秒自动重连

四、推荐算法详解

4.1 架构

```
浏览器 ──→ POST /recommend (allEntries) ──→ Cloudflare Worker
                                                  |
                                        并行拉取 TMDB (20-40 次请求)
                                                  |
                                        评分排序 → 返回 Top30/60
                                                  |
                                        结果存入 Cache API (1h TTL)
```

4.2 基础模式（<100 部评价）

- **5 颗种子**：3 高分(score≥8) + 2 随机中分(6≤score<8)
- **3 条路由**：推荐 / 相似 / 类型发现
- **评分因子**：类型 40% + TMDB 评分 25% + 源评分 20% + 热度 10% + 新片 5%
- **展示**：显示 Top30 候选，取前 12 部

4.3 高级模式（≥100 部评价）

- **10 颗种子**：5 高分 + 3 中分 + 2 避雷(score<4)
- **4 条路由**：推荐 × 2 页 / 相似 / 类型发现 / 导演搜索
- **用户画像**：Top30 高分作品 → 导演排行 + 年代偏好
- **避雷机制**：避雷种子的类型候选被惩罚降权（最高 0.15）
- **评分因子**：类型 35% + TMDB 20% + 源评分 15% + 年代 15% + 热度 10% + 新片 5%
- **展示**：显示 Top60 候选，取前 24 部

4.4 新鲜度保证

- 每次刷新至少替换 75%（基础 9/12，高级 18/24）
- 循环逻辑：追踪评价总数，连续 3 次刷新无变化→第 4 次循环重置

五、数据模型

5.1 Supabase 表

```
entries（评分主表）
├── id              UUID
├── user_id         UUID（FK → auth.users）
├── type            movie | series
├── title           text
├── year            int
├── tmdb_id         int（nullable）
├── poster_path     text
├── director        text
├── total_score     float（加权总分）
├── story           float
├── character       float
├── visual          float
├── editing         float
├── sound           float
├── emotion         float
├── comment         text
├── created_at      timestamp
└── updated_at      timestamp

season_ratings（剧集分季评分）
├── id              UUID
├── entry_id        UUID（FK → entries）
├── user_id         UUID
├── season_number   int
├── season_name     text
└── total_score     float

profiles（用户设置）
├── user_id         UUID（PK）
├── display_name    text
└── updated_at      timestamp

invite_codes（邀请码）
├── code            text（PK）
├── created_by      UUID
├── created_at      timestamp
└── expires_at      timestamp
```

5.2 RLS 策略

- entries：用户可读写自己的记录；所有人可读（用于合并卡片）

- profiles：用户可读写自己的设置；所有人可读
- invite_codes：仅 fank1ng/ceci 可创建

六、Worker 端点

端点	方法	功能	缓存
`/health`	GET	健康检查 + 缓存统计	—
`/tmdb/*`	GET	TMDB API 代理（自动注入 Key）	内存 24h
`/recommend`	POST	推荐算法计算	Cache API 1h

请求示例

```
POST /recommend
{
  "entries": [
    {"user_id":"xxx","tmdb_id":550,"type":"movie","total_score":8.5},
    ...
  ],
  "userId": "xxx"
}

Response:
{
  "movies": [{...}, ...],  // Top30/60 候选
  "totalRated": 120,
  "cached": false          // Cache API 命中时 true
}
```

七、缓存策略

```
第一层：tmdbCache (Worker 内存 Map)
  ├── 电影详情 24h / 推荐/相似 24h
  ├── 搜索/热门/发现 1h
  └── 最大 500 条，FIFO 淘汰

第二层：Cache API (Cloudflare 边缘缓存)
  ├── /recommend 结果 1h
  ├── 按 userId + 评分哈希 缓存
```

```
  └── 评分变化自动失效

第三层：前端 discoverCache
  ├── recommend / week / toprated
  └── 会话级，切换标签不清空
```

八、限流分析（20 人规模）

资源	限额	实际用量	风险
TMDB API	40 次/10s	缓存命中 0，首次 20–40 次	低
Cloudflare Worker	100,000 次/天	~1,000 次/天	无
Supabase	500MB DB + 50,000 月活	~100MB + 2 用户	无
GitHub Pages	100GB/月 带宽	~10MB/月	无

九、文件结构

```
filmnote/
├── index.html          # 主应用（3300 行 SPA）
├── movie-rater.html    # 重定向 → index.html
├── CHANGELOG.md        # 更新日志
├── .gitignore
├── worker/
│   ├── src/index.js    # Worker 代码（TMDB 代理 + 推荐引擎）
│   ├── wrangler.toml   # Cloudflare 配置
│   └── package.json
└── .git/
```

十、关键技术决策

决策	理由
原生 JS，无框架	2 人项目，零构建步骤，GitHub Pages 直接部署
Supabase BaaS	免费 PostgreSQL + Auth + RLS + Realtime，无需自建后端
Cloudflare Workers	$0 实现 TMDB 代理 + 推荐算法服务端化
单文件 SPA	无打包工具链，修改即上线

合并卡片	减少条目重复显示，直观对比两人评分
服务端推荐	减少浏览器并发 TMDB 请求，Cache API 跨实例缓存